Joe Pan — Senior iOS Engineer

shinren.pan@gmail.com · github.com/shinrenpan · App Store
shinrenpan.github.io · YouTube

SUMMARY

Over **15 years** of iOS development, covering MFi hardware accessories, instant messaging, video streaming and fintech apps, from pure Objective-C through to Swift and SwiftUI.

I spend most of my time on **architecture**, and I start from what UIKit, SwiftUI and Swift Concurrency were actually designed to do rather than from a third-party framework that works against them. Dependencies follow the same rule: if an Apple API can do the job, it gets used, which keeps maintenance risk and binary size down. I'm not attached to any one architecture and can work inside a team's existing conventions.

Most common app feature types have come up at some point over the years, so I can usually work out where a problem is coming from fairly quickly.

Outside of work I've built and shipped three apps to the App Store on my own, handling design, development, backend integration and release. Recent side work has gone into HL7 FHIR, including a FHIR R4 server written in Swift.

SKILLS

Languages

Swift 3.0 – present · Objective-C 2009 – present

UI

Both UIKit and SwiftUI, including complex custom interfaces built from scratch. The two are combined as the basis of MVVMC, with `UIHostingController` as the navigation unit and SwiftUI handling rendering. Early work with cocos2d for iPhone led on to SpriteKit, which is where animations that UIKit and SwiftUI can't do natively end up.

UIKit · SwiftUI · UIHostingController hybrid · AutoLayout · Core Animation · MapKit · SpriteKit

Architecture

Led or contributed to architecture adoption and refactors on several teams; designed a custom one (MVVMC) when the standard options weren't enough.

MVC · MVVM · Clean Swift · TCA · MVVMC (custom) · @Observable / @MainActor

Swift Concurrency

Took over an existing Swift 5+ codebase, introduced Swift Concurrency and refactored it to Swift 6 readiness.

async/await · @MainActor · actor · Sendable · Task / TaskGroup

Networking

URLSession by default, gRPC where the project needs it. Also practical experience with Alamofire and other third-party networking libraries.

URLSession · gRPC (grpc-swift / connect-swift) · Alamofire

Real-Time Communication

Shipped projects using WebSocket, XMPP, MQTT and SignalR, plus live video streaming.

WebSocket · XMPP · MQTT · SignalR · Live Streaming (HaishinKit / ijkplayer / ReplayKit)

Persistence

Native options first, third-party where the project already called for it.

CoreData · SwiftData · Keychain · NSKeyedArchiver · Realm · fmdb

Backend

Wrote a complete HTTP server in Swift (see Siming under Projects); also practical experience with Python FastAPI and Docker deployment.

Swift on Server (Hummingbird 2 / SwiftNIO) · PostgresNIO · PostgreSQL · Python FastAPI · Docker

Healthcare Integration

Built the FHIR stack end to end in self-directed projects: the server (Siming), a SMART on FHIR client (Tideng), a TW Core IG model package (TWCoreFHIRModels) and an appointment app (FHIRpass). Also a working picture of Taiwan's healthcare IT reality: the mainstream HIS vendors ship no native FHIR export, and the trade-off between SMART on FHIR and static Bearer Tokens.

FHIR R4 | SMART on FHIR | TW Core IG | HAPI FHIR | Apple FHIRModels | TWCoreFHIRModels | ASWebAuthenticationSession

Bluetooth

Built serverless device-to-device links with CoreBluetooth (the SideBell call bell); earlier work covered MFi hardware accessories and BLE integration.

CoreBluetooth | iBeacon

Toolchain

Category	Tools
Version Control	Git
Package Management	Swift Package Manager, CocoaPods, Carthage
Project Management	XcodeGen, Tuist
CI / CD	Xcode Cloud, Jenkins, Fastlane, Firebase App Distribution
Spec-Driven Development	Spectra (SDD): capability specs and change proposals kept under version control, written before the code
AI Collaboration	Claude Code (constrained by CLAUDE.md + Skill documents so output matches the project's existing architecture)

EXPERIENCE

DRACO EVOLUTION

Apr 2025 – Jun 2026

Senior iOS Engineer Took over from an outsourced team; sole iOS engineer at the company

Assessed code quality, paid down technical debt and led the architectural changes.

- **TCA upgrade and removal:**Inherited the project at TCA 1.15.2 and upgraded incrementally to 1.22.2. While maintaining the app and adding features, found that TCA's built-in Navigation Stack couldn't handle the app's non-standard routing rules, started a gradual rewrite, and removed TCA on 18 Sep 2025 once it was stable.
- **Custom MVVMC architecture (GitHub):**After removing TCA, SwiftUI's native NavigationStack turned out to have the same problem with complex non-standard routing (dismissing a checkout flow, switching tabs and clearing a navigation stack at the same time). That, plus not wanting the whole codebase's architecture tied to a third-party package, led to designing MVVMC in native Swift, borrowing ideas from TCA and MVI: MVVM with a HostController layer (C) added on top. M handles State / Domain Models / DTOs, VM uses `@Observable @MainActor` with a single entry point (`doAction`) for business logic, V is pure SwiftUI with no navigation logic, and C (`UIHostingController`) is the sole Router, with all navigation going through `AppRouter.shared`. Later open-sourced with an MCP Server so Claude Code can pick up the spec in any project.
- **Swift 5 → Ready for Swift 6 migration:**The inherited codebase was Swift 5+; introduced `async/await`, `@MainActor`, `actor` and `Sendable` over time, finishing the Swift 6 compatibility work alongside the TCA removal and MVVMC refactor.
- **Xcode Cloud CI/CD pipeline:**Dev builds go to TestFlight internal testers and production builds to TestFlight external testers, with no manual steps.
- **US App Store release:**Published exclusively on the US App Store to comply with SEC regulations; subsequently taken down following an API shutdown.
- **AI-assisted development workflow (Claude + Skill):**Wrote `CLAUDE.md` and a set of Skill documents (SwiftUI, ViewModel, HostController, Model, Swift Concurrency and so on) so Claude produces feature code that follows the project's architecture.

Evervictory Technology

Sep 2023 – Sep 2024

iOS Engineer The main developer on a 3-person iOS team; the other two focused on an in-house K-Line library and a separate VPN app

Primary maintainer of a fintech trading platform and JinTian GT (a precious-metals trading app), plus contributions to an in-house IM client built on Tinode.

Fintech Trading App (demo)

- Introduced XcodeGen to eliminate persistent `.xcodeproj` merge conflicts in a multi-developer workflow.
- Integrated SignalR for real-time quote streaming; resolved UICollectionView cell-update jank during scroll using `CADisplayLink + RunLoop`.
- Replaced UITableView with UICollectionView across the codebase to enable richer layouts and more flexible cell composition; maintained and extended the in-house K-Line charting library.
- Wrote shell scripts to switch between dev / QAT / production environments; implemented in-app live language switching.

JinTian GT App (demo)

- Swift / Objective-C mixed-language development; integrated gRPC for data transport.
- Applied the Rate Limiting UITableView/UICollectionView Reloads technique to throttle excessive reloads triggered by high-frequency gRPC updates.
- Implemented a review-mode UI-hiding mechanism to pass App Store review.
- Built a CI/CD pipeline with Jenkins + Fastlane; used Firebase App Distribution for dev / QAT builds.
- Adopted Xcode 15 asset symbol generation; built a JavaScript bridge for bidirectional HTML5 ↔ native communication.

IM Client (Tinode-based)

- Dropped Storyboard for fully programmatic UI, since Tinode's default interface needed heavy customisation.
- With no Golang engineer on the team, learned Go and read through the Tinode server codebase to understand how the server behaved and how far the IM features could go.

Heng Yuan Technology Co., Ltd.

Apr 2022 – Jul 2023

iOS Engineer Sole iOS engineer on the project (demo)

Took over an Objective-C codebase and led the technical modernisation of an app covering short video, long video, comics, novels, games, live streaming and chat.

- **Led ObjC → Swift migration:**Took a gradual approach: all new features in Swift, legacy modules refactored bit by bit.
- Introduced XcodeGen to eliminate Git project-file conflicts; researched Tuist and evaluated both tools.
- **Refactored local HLS playback architecture:**The original approach downloaded m3u8 files locally and served them through GCDWebServer, which was expensive to maintain. Replaced it with a custom `AVAssetResourceLoaderDelegate`: `file://` is swapped for a custom `local://` scheme so AVPlayer hands the request to the loader, which swaps back to `file://` and reads the local data. This removed the GCDWebServer dependency and brought binary size down (reference article).
- Built the comic reader, novel reader and embedded game pages, all using WebView with bidirectional native communication.
- Integrated a live-streaming SDK written in Flutter, resolving Flutter module embedding issues and debugging across framework boundaries.
- Coordinated with a third-party signing vendor for enterprise distribution outside the App Store.

Gamania Group

Jun 2019 – Feb 2022

iOS Contract Engineer

BeanFun App

- Collaborated with the team to introduce Clean Swift architecture, progressively migrating Objective-C modules to Swift.
- BeanFun loads most of its features in WebViews and needs hot-reload support; built the WebView ↔ native bidirectional communication, including JavaScript bridge design, event propagation and page state synchronisation.

Gamania Group

Sep 2018 – Mar 2019

iOS Contract Engineer Primary iOS developer

In-House Enterprise App "teamup!"

Internal project management and approval platform supporting instant messaging, digital sign-off, and access control. Used by Gamania's CEO during an Antarctic expedition as a remote management tool (press coverage).

- Optimised for low-bandwidth conditions: cached API responses with fmdb for offline use, and built multi-tier image caching (show the thumbnail first, download the full image in the background and swap it in) so the app stayed usable on slow connections.

Wistron ITS Aug 2016 – Mar 2018

iOS Engineer On-site at Cathay Life Insurance

Wistron's only dispatched iOS engineer on site, and the maintainer of the Cathay Life Insurance app. The codebase was Objective-C; following Cathay's internal decision to adopt Swift, took responsibility for implementing new features in Swift and progressively migrating legacy Objective-C modules.

Hoter Information Mar 2016 – Jun 2016

iOS Engineer

Developed an in-house hotel workforce management app for staff scheduling, real-time reporting, and check-in. The backend used SignalR for chat, an uncommon choice at a time when XMPP and MQTT dominated, which is where the SignalR experience comes from. Also implemented geofenced check-in using Core Location.

Internet Mobile Technology Apr 2015 – Jan 2016

iOS Engineer

Built a shopping app centred on a gamification concept inspired by Tap Titans: users earn virtual currency or discount coupons through in-app gameplay, redeemable in the store. Implemented chat with XMPP and built the game module with SpriteKit, the first time integrating a game engine into an e-commerce app.

Earlier Experience (2010 – 2015)

Company	Period	Key Work
PiPiMy	Jan 2015 – Mar 2015	C2C second-hand trading app (Beta)
Timeline Technology	May 2012 – Jun 2014	Template-based storefront app framework / coupon app
JamZoo	Nov 2012 – Jun 2013	Migrated "Single Bank" app chat from timer polling to MQTT (following Facebook's adoption of MQTT), resolving lag; built offline-capable car rental app and HTML5 e-book WebView
Viamedia Mobile Corporation	Sep 2010 – Feb 2012	Media Player interactive app, mini-games, BLE
Sunplus Semiconductor	Mar 2010 – Jul 2010	MFi hardware accessory app, internet radio, local music playback

PROJECTS

Shipped Apps

HerbMeet App Store

A crowdsourced vegetarian and vegan restaurant map for Taiwan. A restaurant can carry several tags at once (vegan, lacto, ovo, five pungent ingredients), and closed venues turn grey instead of disappearing. Corrections only take effect once **distinct accounts** have voted them through, so no single account can rewrite an entry. Swift 6 with SwiftUI and MapKit, backed by Supabase for data and voting logic. Localised in Traditional Chinese and English, free to use, with a one-time IAP for extra voting weight.

FoodEntropy App Store · GitHub

A food-expiry tracking app, actively maintained, and MVVMC running in a real shipped product. The widget reuses the app's own presentation code rather than a second UI layer; persistence is SwiftData with iCloud sync left as a switch the user can flip, so the schema stays within what CloudKit will accept. Built with Spectra's SDD workflow: features start as capability specs under `openspec/specs/`, changes start as proposals and are archived once implemented, with specs and proposals under version control.

SideBell App Store · GitHub

An accessible call bell that works without the internet. The patient taps once on an iPad; the caregiver's iPhone raises an alarm, says out loud what is needed, and keeps repeating until someone responds — all over a direct Bluetooth link, with

no server, no account and no subscription. Built for people with ALS, severe motor impairment or anyone recovering in bed. Pure SwiftUI on the MVVMC architecture, 107 tests, with a demo video and a bilingual set-up guide.

Architecture and Open Source

MVVMC family MVVMC · MVVMR · MVVMC-Skip

The four-layer architecture designed and put into production at DRACO (described above), packaged as an open-source repo with a runnable Demo and an MCP Server.

It later went two ways: **MVVMR** reimplements it in pure SwiftUI, with M and VM unchanged and navigation moving to `NavigationStack` plus an `@Environment` AppRouter, so routes become values rather than commands. **MVVMC-Skip** carries it to Android through Skip.tools, swapping the C layer behind `#if SKIP` with zero changes on the iOS side.

WebParser GitHub

A Swift Package for web parsing built on WKWebView off-screen rendering, for dynamic pages that need JavaScript to run (SPAs, dynamic DOM). It exposes a type-safe generic Mapper interface (JSON → `Decodable`, custom Regex extraction) and is Swift 6 strict concurrency compliant. Swift Testing, GitHub Actions CI, distributed via SPM.

Healthcare / FHIR

Siming GitHub

A FHIR R4 server written in Swift, aimed at clinical data for small clinics and compliant with TW Core IG. Hummingbird 2 (SwiftNIO) + PostgresNIO (no ORM) + Apple FHIRModels, covering 24 FHIR R4 resource types with CRUD, search, history, compartment and transaction bundle, plus a built-in resource browser, one-command Docker deployment and GitHub Actions CI. Terminology validation is deliberately out of scope, delegated to an optional HL7 Validator sidecar.

Tideng GitHub

An iPad-native SMART on FHIR client using the standard SMART standalone launch, verified against the public `launch.smarthealthit.org` sandbox and the Siming + Keycloak environment bundled with the project. Implements the resources a clinical browsing flow needs: `Patient`, `Encounter`, `MedicationRequest`, `Observation`, `Practitioner` / `PractitionerRole`.

FHIRpass GitHub

A healthcare appointment app MVP where patients book and staff process the bookings. It is built around the two things that actually block third-party services from reaching a hospital — the security perimeter and personal data compliance — so it splits into three tracks that each work on their own: an offline QR path, a SMART on FHIR authorization path (`ASWebAuthenticationSession` + OAuth2 + PKCE, token locked into the Keychain) and an iPad counter that scans and writes to FHIR. I built the iOS app, the FastAPI service, the HAPI FHIR deployment and the scanning frontend.

TWCoreFHIRModels GitHub

A Swift Package adding strongly typed TW Core IG extensions on top of Apple's FHIRModels. Producing FHIR resources that meet Taiwan's Ministry of Health and Welfare specs straight from `ModelsR4` means looking up Profile and CodeSystem URLs by hand, hardcoding them, and writing your own field validation. This package adds a `.twCore` namespace API so Taiwan-specific fields can be set directly, with `validateTWCore()` checking the SHALL-required fields.

- Strongly typed `.twCore` namespace — assign ID card number, declare Profile, validate SHALL fields without any hardcoded URLs
- Swift 6.2 strict concurrency, GitHub Actions CI, distributed via SPM

Earlier open-source work (Objective-C era)

Open-source components released during the 2014–2018 Objective-C years; `SRPPlayerViewController`, a minimal ijkplayer-based player, picked up around 30 stars. The rest are on GitHub.

EDUCATION

Institution	Programme	Period
Chaoyang University of Technology	Information Management (evening programme)	Sep 2004 – Jan 2007
Institute for Information Industry	iOS Development (6-month programme)	2009

> *Graduation project: developed rhythm game *Block Ten* using cocos2d for iPhone, winning **Gold Award at the 2009 Digital Content Creation Competition (Mobile Game category)**. After graduating, contracted with classmates to build a music app.*

Languages: Mandarin Chinese (native) English (technical reading and writing)